06 — Implementation Plan

Product: EventPilot — The Living Plan **Rule:** Inspect the existing repository and official challenge requirements before changing files.

Phase 0 — Verify and inspect

- [ ] Read the official challenge brief, deadline, submission requirements and restrictions.
- [ ] Inspect repository, existing app, package manager, framework, scripts and environment configuration.
- [ ] Capture baseline build/lint/test output.
- [ ] Identify existing working functionality; do not overwrite it blindly.
- [ ] Confirm Gemini API/model support, quotas, pricing, terms and server-side integration approach from official documentation.

Gate: No architecture decision based only on assumptions.

Phase 1 — Plan and design

- [ ] Finalize the MVP and deferred scope.
- [ ] Confirm data types, budget rules, scheduling rules and state transitions.
- [ ] Define the design tokens and responsive shell.
- [ ] Add `.env.example` with placeholder variable names only.

Gate: PRD, TRD, flow, UI brief and schema agree.

Phase 2 — Build the deterministic core

- [ ] Event setup form and validation.
- [ ] Sample plan that works without an API key.
- [ ] Budget engine with integer minor units and pure functions.
- [ ] Four required planning areas.
- [ ] Task/date logic and dependency validation appropriate to MVP.
- [ ] Unit tests for valid and invalid edge cases.

Gate: Budget and date tests pass; no AI dependency for basic demo.

Phase 3 — Replanning

- [ ] Support guest count, event date and total budget changes.
- [ ] Generate typed change records.
- [ ] Build before/after review.
- [ ] Implement accept/reject and locked-item protection.
- [ ] Add tests for rejection, acceptance and locked items.

Gate: No change is silently committed.

Phase 4 — AI integration

- [ ] Add server-side AI endpoint only if allowed and feasible.

- [ ] Keep keys server-side.
- [ ] Validate input and structured output.
- [ ] Add timeout, bounded retry and rate-limit handling.
- [ ] Keep deterministic fallback and sample data.
- [ ] Test malformed response, provider failure and unavailable key.

Gate: AI failure does not break the plan.

Phase 5 — UX and accessibility

- [ ] Responsive mobile/tablet/desktop layouts.
- [ ] Keyboard-only core workflow.
- [ ] Focus states, labels, status announcements and error association.
- [ ] Loading, empty, success and failure states.
- [ ] No horizontal overflow at target viewports.

Gate: Accessibility and responsive smoke tests pass.

Phase 6 — Quality and release

Run the actual scripts discovered in the repository; do not assume script names.

- [ ] Install dependencies from the lockfile.
- [ ] Run formatter/linter.
- [ ] Run typecheck.
- [ ] Run unit and integration tests.
- [ ] Run production build.
- [ ] Run end-to-end smoke test if available.
- [ ] Inspect browser console during core journeys.
- [ ] Inspect network requests; resolve essential failures.
- [ ] Check dependency vulnerabilities and secrets.
- [ ] Verify deployment and submission link.
- [ ] Record commands, outcomes, and any remaining warnings/errors.

Gate: No known blocking errors; no known console errors in tested core journeys; no known failed essential requests; no avoidable warnings; no unresolved critical security findings.

Priority if time is short

1. Complete four required planning areas.
2. Make budget math correct and testable.
3. Make one supported change-and-review workflow reliable.
4. Ensure sample mode works without external services.
5. Add AI personalization.
6. Polish visual design and advanced scheduling last.

Definition of done

A user can create or load a plan, inspect budget/timeline/vendor checklist/ideas, change a supported constraint, review the impact, accept or reject changes, and continue using the app. The build and required tests pass, and verification results are documented honestly.